

PROJECTION MAPPING SDK

Engine-Agnostic Projection Mapping in Modern C++20

Warping, edge blending, calibration and geometry math as one standalone native library — no rendering, no engine code, one stable C ABI for every host.

v1.0.3

C++20

Windows x64

Unity · Unreal · Native



github.com/vinayak-vc/ProjectionMappingC

Release 1.0.3 · github.com/vinayak-vc/ProjectionMappingC/releases/tag/1.0.3

One SDK, Every Engine



What it is

- Core computation of professional projection mapping tools — the Resolume / MadMapper / TouchDesigner class
- Standalone C++20 library: warping, edge blending, calibration, mesh & geometry math
- Stable C API — opaque handles + status codes — built for P/Invoke and FFI
- Ships as DLL / .so / .dylib or static lib



What it is not

- Not a Unity project or an Unreal plugin — those consume the SDK via thin wrappers
- No rendering: zero DirectX / OpenGL / Vulkan, no GUI, no engine includes
- No algorithms in C# or Blueprints — computation always stays native
- Everything graphical belongs to the host application

CONSUMERS

Unity (P/Invoke)

Unreal (plugin)

Godot

Qt

Native C++

C#

Rust

Python — planned

Quality bar: commercial-grade — 90% test coverage target, Doxygen on all public functions, thread-safe subsystems, no crashes across the API boundary.

Nine Modules, One Library



Core

Lifecycle Context, logging facade, error codes, typed config, versioning, handles



Math

Vector2/3/4, Quaternion, Matrix4, Transform, BoundingBox, Ray, Plane — constexpr



Geometry

Mesh + builder + subdivision, BVH, KD-tree, ray intersection, Bezier / spline, UV



Projection

Projection mesh, surface & camera, projection matrix, homography



Warp

Grid / Bezier / bicubic / perspective / mesh warp, WarpNode graph, vertex editing



Blend

Blend regions & curves, gamma, black level, LUT + mask generation



Calibration

Camera / projector calibration, chessboard, ArUco, AprilTags, GrayCode, pose



Serialization

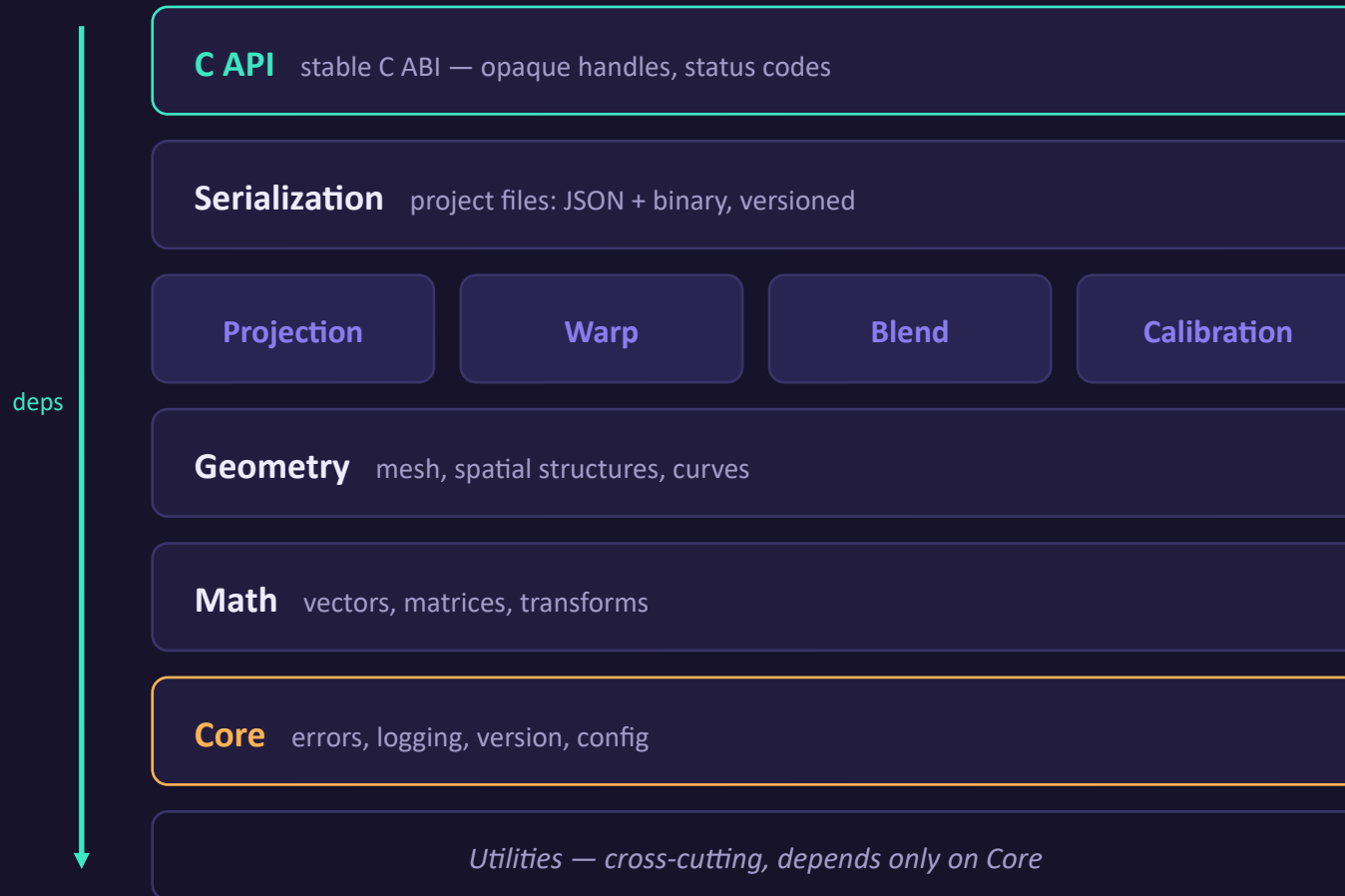
JSON + binary project files, versioned — WarpNode, Mesh, BlendConfig



Utilities

File IO, thread pool, profiling, timers, memory pool — cross-cutting

Layered — Dependencies Point Down Only



Hard rules

- Core never depends on higher modules — no circular dependencies, ever
- Projection / Warp / Blend / Calibration may use Geometry, Math, Core, Utilities — never each other
- OpenCV confined to Calibration; GLM, spdlog, nlohmann-json never appear in public headers
- Public headers expose only std types and PMSDK types
- Only PMSDK_API symbols exported; hidden visibility elsewhere

The C API Is the Contract

```
consumer.c

#include <PMSDK/PMSDK_C.h>

pmsdk_mesh_t*      mesh = pmsdk_mesh_create();
pmsdk_warpline_t* node = pmsdk_warpline_create();
pmsdk_mesh_t*      out  = pmsdk_mesh_create();

/* populate vertices / indices ... */

if (pmsdk_warpline_process_mesh(node, mesh, out)
    == PMSDK_SUCCESS) {
    /* warped vertices ready for host GPU */
}

pmsdk_warpline_destroy(node);
pmsdk_mesh_destroy(out);
pmsdk_mesh_destroy(mesh);
```



Opaque handles

Every object lives behind *_create() / *_destroy() — no C++ types, no STL at the boundary



Status codes + last error

Every call returns pmsdk_status_t; PM_GetLastError() gives per-thread detail — errno style



Exceptions never escape

C++ may throw internally; all exceptions caught and translated at the C boundary — never crash



Version handshake

PM_GetVersion() at runtime vs kHeaderVersion at compile time — drift breaks the build, not the user



C++ headers = convenience

Native consumers may use pmsdk:: classes, but ABI stability is promised only for the C API (D-006)

Design Decisions That Shape the Code



Generation-checked handles (D-010)

64-bit handle = 32-bit generation + 32-bit slot. Generation bumps on remove, so stale handles fail lookup instead of aliasing reused slots; shared_ptr storage keeps concurrently-removed objects alive.



PImpl + per-method export (D-009, D-015)

STL containers hidden behind PImpl; each public method exported individually instead of dlexport-ing whole classes — dodges MSVC C4251 under /W4 /WX.



Thread-local last error (D-011)

errno-style PM_GetLastError backed by function-local thread_local Status — the single sanctioned exception to the “no static mutable state” rule.



Hand-rolled constexpr math (D-012)

Public Vector / Matrix / Quaternion are plain value types — no glm in public headers, no PImpl overhead on hot math, 16-byte aligned and SIMD-ready.



Fixed vertex layout (D-014)

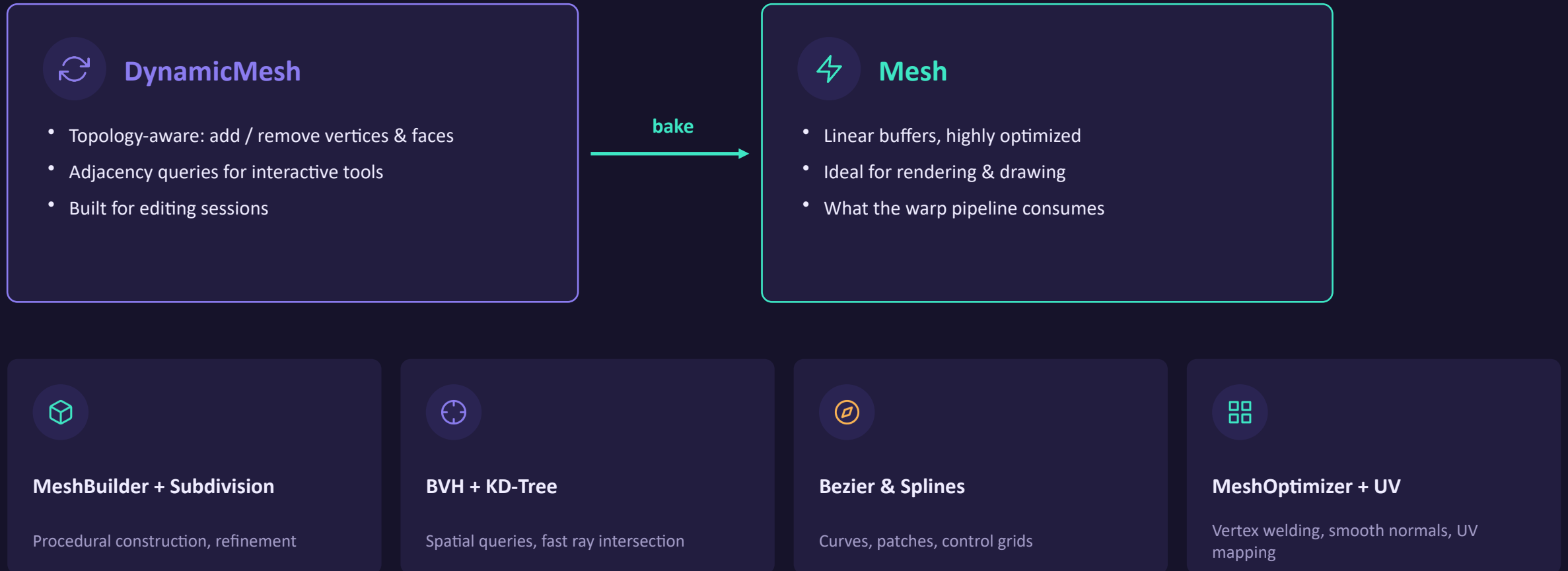
Vertex = interleaved Position, Normal, UV, Color. Cache-friendly, covers ~99% of projection mapping use cases without flexible-format complexity.



No globals, no singletons

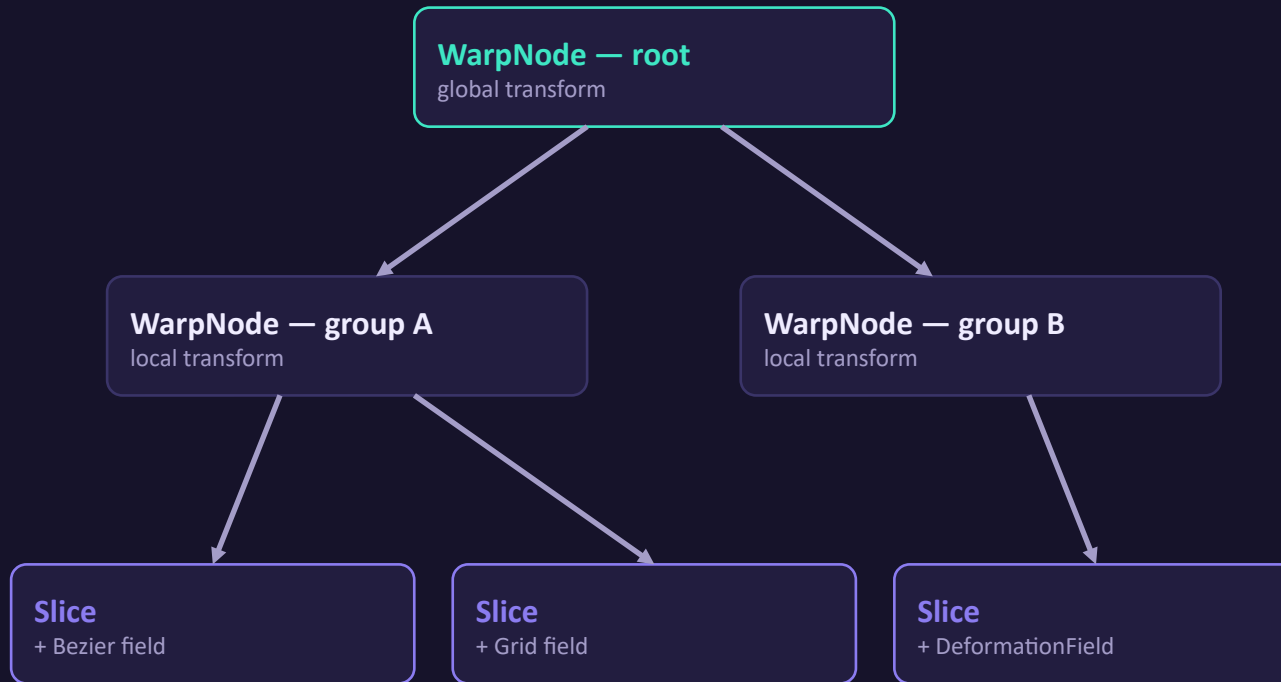
A Context object owns Logger + Config; RAII everywhere, thread-safe subsystems, std::span and smart pointers throughout.

Two Meshes: Edit Freely, Render Fast



Users manipulate the DynamicMesh in tooling, then bake to an immutable Mesh for the render path (D-016).

A Scene Graph of Warped Surfaces



Mirrors the slice-grouping model of Resolume Arena and MadMapper (D-017).



How warping works

- Every node carries a local transform; groups apply spatial transforms on top of child deformations
- Leaf nodes attach an optional DeformationField — Bezier patch or grid warp
- Projector maps content through the hierarchy; Sampler evaluates warped UVs
- Interactive vertex editing API with control grids — undo / redo ready
- `pmsdk_warpline_process_mesh()` deforms an input Mesh into an output Mesh, fully native

Seamless Multi-Projector Output

Two projectors, one wall



- Soft-edge gradients with configurable blend curves per region
- Gamma-correct falloff — perceptually linear crossfade
- Black-level lift so overlap zones match single-projector black
- Luminance compensation across heterogeneous projectors



What the SDK emits

- BlendConfig — declarative description of regions, curves, gamma and black level
- Generated LUTs the host samples in its own shader
- Generated masks per projector for GPU composite
- EdgeBlend + MaskGenerator run fully on the CPU side — the SDK never touches your GPU

BlendConfig is exposed through the C API (D-018), so hosts can preview and tweak interactively.

From Real-World Surface to Digital Twin



Camera & projector calibration

Intrinsics + extrinsics via OpenCV — chessboard, ArUco and AprilTag targets



Structured light (GrayCode)

Pattern generation, capture and decode — GrayCodeDecoder + triangulation (M17)



Homography solver & pose estimation

Plane-to-plane mapping and full 6-DoF projector pose



Read-back for tooling

Warped geometry readable through the C API for engine-agnostic previews (D-018)



Direct camera capture (D-019)

Instead of the host engine grabbing WebCam frames, allocating Texture2D memory on the main thread and marshalling them across, the GrayCodeDecoder binds straight to the physical camera with cv::VideoCapture.

Result: memory and frame extraction stay entirely off the host main thread — zero-lag calibration inside Unity and Unreal.

OpenCV is confined to the Calibration module — no other module, and no public header, ever sees it.

End-to-End: Host Frame to Warped Output

1

Create Context

Host (Unity / Unreal / native) opens the SDK through the C API — opaque handles, status codes



2

Build geometry

MeshBuilder or loaded asset; edit as DynamicMesh, bake to render Mesh



3

Assemble WarpNode graph

Group slices; per-node transform plus optional DeformationField (Bezier / grid)



4

Warp processing

pmsdk_warpnode_process_mesh() deforms vertices — SIMD + multithreaded native C++



5

Blend & masks

Edge-blend LUTs and per-projector masks generated for the host GPU composite



6

Read back & render

pmsdk_mesh_get_vertices() returns warped geometry — host draws it with its own renderer



Save / load anywhere

The whole state — WarpNode hierarchy, meshes, BlendConfig — serializes to versioned JSON or binary project files at any step, and round-trips back.



One contract, any host

Steps 1–6 look identical from Unity C#, Unreal C++ / Blueprints, or a plain C program — the C ABI is the same everywhere; only the thin wrapper changes.

GrayCode Calibration, Start to Finish



The decoded mesh feeds straight into the WarpNode graph

One calibration pass replaces manual corner-pinning: the triangulated surface becomes the projection target, and the warp engine aligns content to it. In Unity this is one click — Calibration menu → Start Calibration.

UPM Package + Guided Setup Wizard

1

Install

Package Manager → “Add package from disk...” → select com.viitorx.pmsdk/package.json
— native binaries import automatically

2

Run the Setup Wizard

Tools → Projection Mapping → Setup Wizard; configure projector + physical camera on-screen

3

Scene generated

Wizard creates the DeformationField, WarpNode and Mesh for your scene — zero manual wiring

4

Optional: GrayCode calibration

Calibration menu → pick camera → Start Calibration; the SDK builds a pixel-perfect surface mesh

```
ProjectionMapper.cs

using PMSDK;

// 1. Create a Warp Node
IntPtr warpNode =
    NativeBindings.CreateWarpNode("MyProjector");

// 2. Load a 3D mesh
IntPtr mesh =
    NativeBindings.LoadMesh("path/to/mesh.obj");

// 3. Process — runs entirely in native C++
NativeBindings.ProcessMesh(warpNode, mesh);
```



Multithreaded native core — the Unity main thread never blocks; C# is a thin P/Invoke wrapper with no algorithms.

Drop-In Plugin, Blueprints or C++

1

Copy the plugin

Place the ProjectionMapping folder in YourProject/Plugins/ — ProjectionMapping.uplugin at its root

2

Rebuild & enable

Launch the project, accept the module rebuild prompt, enable under Edit → Plugins → Projection Mapping

3

Blueprints

Right-click any Event Graph → search “Projection Mapping” — Create Warp Node, Apply Deformation Field on static or procedural meshes

4

C++ modules

Add “ProjectionMappingSDK” to PublicDependencyModuleNames in Build.cs, include the C-API headers, call the native API directly

MyActor.cpp

```
#include "PMSDK/C_API/WarpAPI.h"
#include "PMSDK/C_API/GeometryAPI.h"

void AMyActor::StartMapping() {
    // Call the native C-API directly
    pmsdk_warp_node_h warpNode =
        pmsdk_warp_node_create("MyProjector");

    // ... setup and projection logic ...
}
```



The plugin dynamically loads ProjectionMappingSDK.dll from Source/ThirdParty/ProjectionMappingSDK/bin/Release/ — keep the bin/ folder when packaging.

Build System, Quality Bar, Milestones



Build & CI

- CMake ≥ 3.25 + Ninja presets (MSVC, Clang, GCC) + VS2022 generator preset
- vcpkg vendored as a pinned submodule — reproducible dependencies, zero global installs
- Deps gated behind manifest features: tests $\rightarrow$ gtest, calibration $\rightarrow$ OpenCV — consumers skip what they don't use
- GitHub Actions matrix: windows-msvc, linux-gcc, linux-clang, macos



Quality bar

- GoogleTest suites mirror the module layout — 90% coverage target, exhaustive math edge cases
- Strict warnings everywhere: /W4 · -Wall -Wextra -Wpedantic -Wshadow -Wconversion; /WX in CI
- Doxygen on every public function; docs generated from source
- M18 performance pass: SIMD + multithreading through the hot paths

20 MILESTONES — ONE AT A TIME, EACH MUST COMPILE, PASS TESTS AND KEEP EXAMPLES BUILDING



● shipped (19 of 20) ○ backlog — M19 Plugin SDK

RELEASE

v1.0.3

ProjectionMappingSDK_v1.0.3_Windows.zip — everything a host team needs, in one download.



Source github.com/vinayak-vc/ProjectionMappingC



Release package github.com/vinayak-vc/ProjectionMappingC/releases/tag/1.0.3



Docs [docs/](#) — overview, architecture, roadmap, decisions + Doxygen API reference

```
package contents

ProjectionMappingSDK_v1.0.3_Windows/
+-- C_API/
|   +-- bin/ProjectionMappingSDK.dll
|   +-- include/PMSDK/
|   +-- lib/ProjectionMappingSDK.lib
+-- Unity/
|   +-- com.viitorx.pmsdk/      (UPM package)
|   +-- README.md              (quick start)
+-- Unreal/
    +-- ProjectionMapping/      (.uplugin)
    +-- README.md              (quick start)
```

Map anything. Any engine. One SDK.